# Repeated-call evaluation of sensor-mapping faults and batch context in LLM-based industrial fault diagnosis

Song Ouyang*

[TODO: Affiliation, city, postcode, country]

**Abstract**

General-purpose large language models (LLMs) are increasingly used to diagnose faults from serialized sensor records. Such interfaces can attach a value to the wrong sensor description, can change a record's label when other records share the request, and can return different answers to identical calls. Single-call benchmarks cannot separate these effects. We propose a repeated-call evaluation that crosses descriptor-mapping faults, meaning-preserving reorderings and context changes with identical repeats, and tests each contrast against call-to-call variation with exact McNemar tests, a crossed case-by-call bootstrap and call-level randomization tests. Two GPT services were evaluated on the Tennessee Eastman Process, C-MAPSS and a measured hydraulic test rig. A cyclic descriptor rotation reduced GPT-5.6 Sol's accuracy by 58.6 percentage points (95% CI 37.1–77.1) and its repeat agreement from $\kappa = 0.98$ to 0.49, whereas GPT-6 Astra returned correct labels under every mapping fault. A random derangement that was highly significant in a single-call test fell within call-to-call variation, and an apparent 12-label position effect was not reproduced under a time-balanced ABBA/BAAB schedule (4.8 points, 95% CI −2.4 to 14.9). On the hydraulic rig, most responses to canonically ordered records collapsed to one label (42% accuracy), whereas reversing the intact fields raised accuracy to 84% (randomization $p = 0.004$). Majority voting removed transient but not systematic errors, and a trusted-channel gateway rejected all visible mapping faults but missed consistent value swaps. Accuracy alone cannot certify an LLM diagnostic interface; repeated, time-blocked calls and independent channel checks are needed.

**Keywords:** fault diagnosis; large language models; sensor data serialization; repeatability; robustness evaluation; Tennessee Eastman Process

## 1 Introduction

Data-driven fault diagnosis has moved from hand-crafted features to deep networks trained on large labelled datasets [1, 2]. Large language models (LLMs) add a different option: a general-purpose service that receives measurements as text and returns a diagnostic label, sometimes with an explanation, without task-specific training [3–5]. The appeal for industrial users is clear. A single interface can serve many assets, domain knowledge about sensors and failure modes can be expressed in words, and the model can be queried by engineers who do not maintain a classifier.

This convenience depends on a serialization layer that is easy to overlook. Each measured value has to be written next to a channel identifier, a name and a unit, and several observations are often packed into one request to reduce cost [6, 7]. Two practical errors follow. A conversion script can attach the description of one sensor to the value of another while leaving every number in place. Separately, a correctly serialized observation can receive a different label because the other records in the request, or its position among them, have changed. The first error alters

*Corresponding author. E-mail: ouyangsong8@gmail.com

the physical meaning of the input; the second leaves the target record intact. Both are routine engineering risks, and both are invisible to an evaluation that reports only accuracy on clean inputs.

Evaluating these errors is complicated by a third factor. Commercial LLM services do not always return the same answer to the same request, even at nominally deterministic settings [8, 9]. When an intervention is tested with one call per condition, any label change can reflect the intervention, the service's call-to-call variation or a change in the service between calls. Related work on tabular and batched LLM inference has shown sensitivity to feature names, column order and batch composition [6, 7, 10, 11], and prompt-format studies have documented large performance swings from superficial changes [12, 13]. Fewer studies separate such effects from identical-input variation on the same cases, and we found none that did so for industrial sensor records.

This paper addresses that gap with a repeated-call evaluation of LLM-based industrial fault diagnosis. The contributions are as follows.

1. **An evaluation framework for LLM diagnostic interfaces.** It specifies three classes of input change (descriptor-mapping faults, meaning-preserving reorderings and context changes), submits every input several times, and tests each contrast at three levels: an exact single-call McNemar test, a crossed case-by-call bootstrap interval and an exact call-level randomization test. Time-blocked ABBA/BAAB and five-block schedules separate input effects from changes in the service over time.

2. **Evidence that mapping faults are service-dependent and invisible to accuracy.** A cyclic descriptor rotation reduced one service's accuracy by 58.6 points and halved its repeat agreement, while a second service gave correct, identical labels under all five mapping faults. Correct labels under a wrong mapping show that accuracy cannot verify channel integrity.

3. **Evidence that single-call tests overstate input effects.** A random derangement with a single-call Holm-adjusted $p < 10^{-6}$ fell within call-to-call variation once all repeats were analysed, and a position effect of 12 of 21 labels (Holm $p = 0.002$) was not reproduced under a time-balanced schedule. The low-accuracy calls behind it were clustered in one early service period.

4. **Validation on measured data and engineering countermeasures.** On a physical hydraulic test rig evaluated in five time blocks, a meaning-preserving reversal of the fields doubled accuracy by preventing whole-request label collapse, so presentation rather than mapping dominated on measured data. A trusted-channel gateway, fixed-panel request construction and majority voting are evaluated with their coverage and cost trade-offs, and tuned supervised baselines show that the services' transfer advantage disappears once classifiers receive target-domain information.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the framework and statistical procedures, and Section 4 the data, services and baselines. Section 5 reports the results, Section 6 discusses their engineering implications and limitations, and Section 7 concludes.

## 2 Related work

### 2.1 LLMs for prognostics, health management and fault diagnosis

Surveys and roadmaps describe how large foundation models could support prognostics and health management (PHM), from data preparation to maintenance decisions, and identify trustworthiness and deployment as open problems [5, 14, 15]. Applied work falls into two groups.

The first adapts a model to the task: vibration features are converted to text and a model is fine-tuned for bearing diagnosis [3, 16, 17], pre-trained LLMs are fine-tuned for complex-system diagnosis [4], reasoning traces are distilled for asset health monitoring [18], or a domain-specific PHM model is trained [19]. The second uses general-purpose services in context. FaultExplainer, for example, asks GPT-4o and o1-preview to explain TEP faults from principal-component contributions and notes that the explanations depend on the supplied variables and can be hallucinated [20], and verification-based pipelines accept only diagnoses whose repeated rationales agree [21]. Benchmarks probe sensor–failure knowledge [22] and coding-based sensor processing [23]. For time series more broadly, LLMs have been reprogrammed or prompted for forecasting [24–26], but ablations show that removing the language model often does not hurt accuracy [27], and strong models reason poorly over raw series [28, 29]. These results make two points relevant here: LLM claims on sensor data need strong conventional baselines, and the in-context, API-only setting that is easiest to deploy is also the one with the least control over the model.

### 2.2 Serializing tabular and sensor records

Feeding non-text data to a language model requires a serialization. LIFT showed that classification and regression tasks can be solved by writing features as sentences [30]; TabLLM compared serializations for few-shot tabular classification and removed or permuted feature names [10]; GReaT encodes rows as "feature is value" text and shuffles feature order during training [31]. Benchmarks and surveys report that table understanding depends on the format, delimiters and input design [32, 33], and a recent study found LLM predictions on tabular data to be sensitive to names, order and number formatting, using identical-prompt repeats as a reference for variability [11]. Industrial records add a physical constraint absent from most tabular benchmarks: each value is meaningful only together with its channel, name and unit, so a wrong association is a data-integrity fault rather than a formatting variant.

### 2.3 Prompt format, order and context sensitivity

LLM outputs change with superficial prompt features. Separators, casing and spacing can move few-shot accuracy by tens of points [12], model rankings change with the instruction template [13], and the order of in-context examples can swing performance between near state of the art and near chance [34]. Majority-label and recency biases explain part of this behaviour [35]. Selection and order biases also affect multiple-choice answers [36, 37] and pairwise judgements [38]. In long inputs, information in the middle is used less than information at the start or end [39], a pattern traced to positional attention bias [40], and irrelevant context can distract a model from the target problem [41]. Batched inference raises the same issues for unlabelled test items: batch size and grouping affect accuracy [6, 42], and item position motivates permutation ensembles [7]. Behavioural testing frames such checks as invariance and directional tests [43]; our mapping faults and reorderings follow that logic.

### 2.4 Non-determinism, repeatability and uncertainty

Hosted LLMs are not reliably deterministic. Identical ChatGPT requests often return different code even at temperature zero [8], and models configured for deterministic output varied in accuracy across repeated runs [9]. Disagreement between samples has also been used constructively: majority voting over sampled reasoning paths improves accuracy [44], and dispersion over repeated answers underlies black-box uncertainty and hallucination measures [45–47]. On the statistical side, McNemar's test is the standard paired comparison of classifiers on the same test items [48], and designs in which items are crossed with other sampled factors require crossed random effects or equivalent resampling [49–51]. In LLM evaluation the repeated call is such a factor, but it is rarely treated as one.

### 2.5 Position of this study

Table 1 compares the closest studies. Each of the individual interventions has precedents. What is missing is an evaluation that applies physical mapping faults, meaning-preserving controls and context changes to the same industrial records, repeats every input, and uses tests whose unit of randomization is the call. The present study provides that evaluation, adds time-blocked schedules to separate input effects from service-period effects, and checks the conclusions on measured data.

**Table 1:** Closest prior studies and the present work. "Repeats" means the same input is submitted more than once and the variation is analysed.

| Study | Input changes | Repeats | Time-balanced | Industrial data |
|---|---|---|---|---|
| TabLLM [10] | feature names removed or permuted | – | – | – |
| Liu et al. [11] | names, order, number format | ✓ | – | – |
| Sclar et al. [12] | prompt formatting | – | – | – |
| Cheng et al.; BatchPrompt [6, 7] | batch size, grouping, position | – | – | – |
| Ouyang et al. [8] | none (identical requests) | ✓ | – | – |
| FaultExplainer [20] | none | – | – | ✓ (TEP) |
| Kiangala and Wang [21] | none (repeated rationales) | ✓ | – | ✓ |
| This study | mapping faults, reorder, background, position | ✓ | ✓ | ✓ (TEP, C-MAPSS, rig) |

## 3 Repeated-call evaluation framework

### 3.1 Serialized diagnostic records

Let $\mathbf{x}_i = (x_{i1}, \ldots, x_{ip})$ be the feature vector of observation $i$ and let $d_j = (\mathrm{ID}_j, \mathrm{name}_j, \mathrm{unit}_j)$ be the descriptor of feature $j$. A serializer $S$ writes each observation as an ordered list of fields $(d_{\pi(j)}, x_{i\pi(j)})$, where $\pi$ is the field order. A request $R = (r_1, \ldots, r_m)$ contains $m$ serialized observations with anonymous IDs, the permitted label set and fixed task instructions. The service returns one label per ID. A response is valid only if it contains exactly one permitted label for every ID in the required order; otherwise the whole request is invalid and none of its labels is scored. Figure 1 summarizes the evaluation workflow.

### 3.2 Input changes

Table 2 lists the input changes. A *mapping fault* replaces the descriptor $d_j$ of each affected position with the descriptor of another channel while keeping the numeric string and its position unchanged, so the channel identity, name and unit move together. We use a one-position cyclic rotation of all descriptors, three fixed random derangements, and a swap of two channels that share a unit (reactor level and separator level, both in percent); the last is the hardest to detect by unit checks. An *intact-pair reorder* permutes whole descriptor–value pairs and keeps every physical association correct; it controls for a change in presentation without a change in meaning. A *context change* keeps the focal records byte-identical and varies either the unlabelled background records in the same request (panel A or B) or the focal block's position before or after the background, with request size held fixed. All permutations and panels were fixed before any inference and are reported individually rather than selected by outcome.

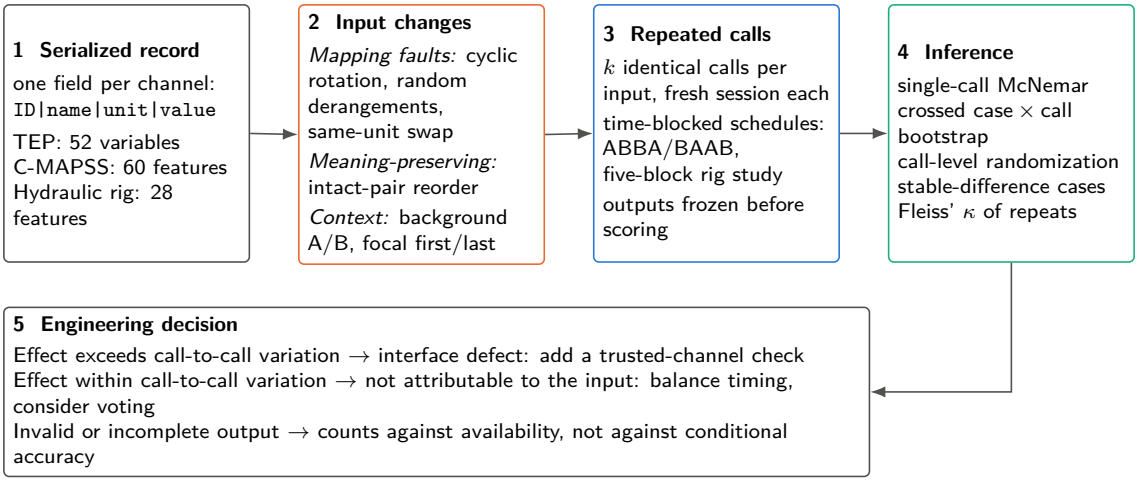



**Figure 1:** Repeated-call evaluation framework. Each input change is crossed with repeated identical calls, and every contrast is tested against call-to-call variation before it is attributed to the input.

**Table 2:** Input changes, the quantities they hold fixed and the question each answers.

| Change | Held fixed | Question |
|---|---|---|
| Mapping fault | Numeric strings and positions | Does the service respond to a wrong physical association? |
| Intact-pair reorder | Every descriptor–value pair | Is the decision sensitive to a meaning-preserving presentation change? |
| Background panel | Focal records, position, request size | Do co-presented unlabelled records change the focal labels? |
| Focal position | Focal records, background, request size | Does the position of the focal block change its labels? |
| Identical repeat | Entire request payload | How large is call-to-call variation? |

### 3.3 Repeated-call design

Every condition is submitted $k$ times as separate calls in fresh sessions. On the source-disjoint TEP panel, $k = 3$ and all 84 calls follow one randomized schedule. Because the order of those calls was random rather than balanced, a condition can by chance be concentrated in one period of the session. Two further studies therefore use time-blocked schedules: an ABBA/BAAB replication of the focal-position contrast (four blocks, two calls per order per block), and a measured hydraulic study in five contiguous blocks, each containing all conditions and two byte-identical duplicates. All outputs are validated and frozen before the answer key is opened.

### 3.4 Statistical inference

For a contrast between a reference condition $L$ and a changed condition $R$ on $n$ shared cases, let $c_{is}^{L} \in \{0, 1\}$ indicate whether call $s$ labels case $i$ correctly, and $\hat{y}_{is}^{L}$ the label itself. Three tests are applied, from least to most conservative with respect to call dependence.

**Single-call McNemar test.** For the prespecified primary call pair (repeat 1), the exact McNemar test compares the discordant counts $b = \#\{c_{i1}^{L} = 1, c_{i1}^{R} = 0\}$ and $c = \#\{c_{i1}^{L} = 0, c_{i1}^{R} = 1\}$ with a Binomial$(b+c, \frac{1}{2})$ distribution [48, 52]. $p$-values are Holm-adjusted within each family of contrasts [53]. This is the test an evaluation with one call per condition would use; it treats all labels in a call as independent.

**Crossed case-by-call bootstrap.** The pooled accuracy difference

$$\Delta = \frac{1}{nk_L}\sum_{i,s} c_{is}^{L} - \frac{1}{nk_R}\sum_{i,s} c_{is}^{R} \tag{1}$$

uses all repeats. Its 95% interval comes from a pigeonhole bootstrap [51]: in each of 10,000 replicates, cases are resampled with replacement and, independently, calls are resampled with replacement within each condition. The interval therefore reflects both case sampling and call-to-call variation. With only three calls per condition the call component is estimated crudely and tends to be too narrow, so the interval is best read alongside the exact test below.

**Call-level randomization test.** Under the null hypothesis that the input change has no effect, the calls of the two conditions are exchangeable within each execution block. We enumerate every reassignment of whole calls to the two conditions that respects the block structure and compute the two-sided $p$-value of $\Delta$ as the proportion of reassignments with $|\Delta^*| \geq |\Delta|$ [54]. Because labels are permuted call by call, the test is valid regardless of the dependence between labels returned in the same call. Its resolution depends on the design: with three unblocked calls per condition there are $\binom{6}{3} = 20$ reassignments and the smallest attainable two-sided $p$ is 0.10, whereas the ABBA/BAAB design admits $6^4 = 1296$ reassignments.

**Label disagreement and repeatability.** For case $i$, let $W_i$ be the mean disagreement $\mathbf{1}(\hat{y}_{is} \neq \hat{y}_{it})$ over all within-condition call pairs and $B_i$ the mean over all between-condition pairs. The excess disagreement

$$E = \frac{1}{n}\sum_{i}(B_i - W_i) \tag{2}$$

is zero in expectation when the calls are exchangeable; a positive value means that labels differ more between conditions than between identical requests. We report a case-bootstrap interval for $E$ and its one-sided call-level randomization $p$-value. A case is a *stable difference* when all calls in each condition agree but the two conditions give different labels. Repeatability of identical calls is summarized by Fleiss' $\kappa$ [55]. Accuracy intervals for single calls use the Wilson method [56].

### 3.5 Engineering countermeasures

Three countermeasures are evaluated. (i) A *trusted-channel gateway* checks every descriptor against a registry keyed by acquisition channel IDs, rejects unknown, duplicate or inconsistent descriptors, and writes accepted records in a canonical order. The acquisition IDs are not passed to the model, so they cannot act as a second, uncorrupted identity. The gateway is tested on clean inputs, intact reorderings, the two visible descriptor faults and a hidden value swap in which a value is moved to another channel together with a consistent descriptor. (ii) A *fixed-panel policy* submits a predefined set of records in canonical order, so that membership and order cannot change between calls. (iii) *Majority voting* over three identical calls, with ties resolved by the first call [44]. Voting and the gateway were assessed post hoc on frozen outputs; neither was used to select results.

## 4 Experimental setup

### 4.1 Datasets and panels

Table 3 summarizes the evaluation panels.

*Tennessee Eastman Process.* We use the expanded TEP simulation data of Rieth et al. [57] for the benchmark process of Downs and Vogel [58], with 52 measured and manipulated variables

and seven classes: normal operation and faults 1, 2, 4, 6, 7 and 13. Training, development and all earlier evaluation runs were excluded globally, and 112 of the remaining 135 run IDs were drawn without inspecting predictions. Seventy runs, ten per class, form the main panel, following the run-level separation recommended for open process-monitoring benchmarks [59]; each contributes one seeded late snapshot (sample index 200–500). Two disjoint background panels, A and B, contain 21 runs each (three per class). A fixed focal subset of 21 main-panel runs, three per class, is used for the context contrasts. The same 70 runs at sample 40, about one hour after fault injection, form the early-stage transfer panel.

*C-MAPSS.* For FD001 and FD002 [60, 61], each engine is summarized over its last 30 available cycles by the mean, population standard deviation, least-squares slope and last value of 15 training-selected channels (60 features). Three retrospective health classes are defined from true remaining useful life (RUL): Healthy (RUL $>$ 80), Degrading (30 $<$ RUL $\leq$ 80) and Critical (RUL $\leq$ 30). FD001 uses 60 held-out engines and FD002 60 test engines, 20 per class, from the six-operating-condition subset. The FD001 representation is applied unchanged to FD002; operating settings are not included.

*Hydraulic test rig.* The UCI condition-monitoring dataset [62, 63] contains 2,205 measured 60-s cycles from one physical rig. The task is internal pump leakage (no, weak, severe). Cooler states occupy three contiguous time segments, so each panel is balanced over all nine pump $\times$ cooler strata. Within each stratum, one middle stable cycle is taken from each of six maximal blocks of constant component conditions, shared valve and accumulator settings are selected by a fixed hash, and three disjoint 18-cycle panels (focal, A and B) are formed. Each cycle is described by the mean and population standard deviation of 14 physical sensors (28 features); the virtual efficiency channels are excluded.

**Table 3:** Evaluation panels. Calls count separate requests to one service; repeated labels of the same case are not independent observations.

| Panel | Classes | Cases | Features | Calls per condition and service |
|---|---|---|---|---|
| TEP late (main) | Normal + 6 faults | 70 | 52 | 3 (7 mapping conditions) |
| TEP focal context | Normal + 6 faults | 21 (+21 background) | 52 | 3 (5 contexts); ABBA/BAAB: 8 |
| TEP early | Normal + 6 faults | 70 | 52 | 3 |
| C-MAPSS FD002 | 3 health classes | 60 | 60 | 3 |
| Hydraulic rig | 3 leakage levels | 18 (+36 background) | 28 | 5 blocks $\times$ 9 conditions |

## 4.2 LLM services and execution

Two services were evaluated in full: `gpt-5.6-sol` (Sol) and `gpt-6-astra` (Astra), both called through the Codex command-line interface (version 0.155.0) with medium reasoning effort. Each call ran in a fresh, empty session with tools, plugins, web access and project context disabled. Requests contained anonymous eight-character IDs, the permitted labels and a short description of each fault mechanism, but no per-case labels, labelled examples or class quotas. Each scheduled job was invoked once with a 900-s timeout; failed or invalid calls were recorded and never replaced. Temperature is not exposed by this interface, and the requested model aliases do not guarantee a fixed server-side snapshot, so all results refer to the recorded call windows.

A preliminary 130-source panel (60 FD001 engines and 70 TEP runs) had evaluated five services with clean and rotated descriptors (Supplementary Sections 3 and 5). There, the rotation reduced primary TEP accuracy by 77.1 points for Sol, 28.6 for GPT-5.5 and 22.9 for Qwen3.8-Max, left Astra's labels unchanged, and caused two of three DeepSeek-V4-Pro

corrupted-TEP responses to exhaust the output budget. These results motivated the source-disjoint study but are not pooled with it.

The source-disjoint TEP study comprised 84 calls (two services, 14 conditions, three repeats) and 4,854 classification slots. The ABBA/BAAB replication added 16 Sol calls. The hydraulic study scheduled 63 Sol calls in five blocks, each with nine grouped conditions plus 18 single-record calls spread across the blocks. Its first run stopped in block R3 after two network time-outs. A versioned completion then executed blocks R3–R5 in full, with byte-identical prompts and the original within-block order, so that every analysed block contains contemporaneous controls. All 37 completion calls were valid. The analysis set combines blocks R1–R2 of the first run with R3–R5 of the completion; the three valid R3 calls of the first run are reported separately (Supplementary Appendix G).

### 4.3 Supervised baselines

Six supervised classifiers were trained with scikit-learn [64]: a random forest (RF) [65], an RBF support vector machine, histogram gradient boosting (HGB), a multilayer perceptron, $k$-nearest neighbours and multinomial logistic regression. They receive the same rounded features and test cases as the LLM but also use labelled training data. Hyperparameters were chosen by five-fold GroupKFold cross-validation on the training data only, grouped by training run or engine and scored by macro-F1. In the *like-for-like* setting, the classifiers are trained on late snapshots of the 100 TEP training runs and on the 100 FD001 training engines, which is the information that was available when the LLM prompts were designed, and are then applied unchanged to every panel. Two further settings give the classifiers target-domain information that the LLM never receives: a *phase-aware* TEP model trained on both late and sample-40 snapshots of the training runs, and FD002 models trained on 260 FD002 training engines after operating-regime normalization (six regimes found by $k$-means on the operating settings, the usual practice for this subset [66]). No test run or engine occurs in any training set. Because the best method differs between panels, we report the range over all six methods rather than a single reference chosen on test scores [27]; per-method results and hyperparameters are in Supplementary Appendix H.

### 4.4 Metrics

We report accuracy, macro-F1, the number of changed labels between paired calls, the statistics of Section 3.4, request validity, token usage and latency. Accuracy is conditional on a valid response; availability is reported separately as valid classification slots over scheduled slots.

## 5 Results

### 5.1 Clean accuracy and transfer

Table 4 compares the two services with the supervised references on clean inputs. On late-stage TEP snapshots both services were almost perfectly accurate, but so were RF and HGB, so this panel shows no advantage for the LLMs. On FD001, tuned HGB reached 85.0% against 71.7% for Astra's primary call (McNemar $p = 0.096$).

The two transfer panels show where the LLMs helped and why. On early TEP snapshots, about one hour after fault onset, every like-for-like reference fell to 22.9–54.3%, while both services stayed at 86–89%; Astra was correct on 24 cases where HGB was wrong and never the reverse ($p = 1.2 \times 10^{-7}$). Trained with early snapshots as well, the references reached 74.3–88.6%, and HGB no longer differed from Astra ($p = 1.00$). On FD002, the like-for-like references reached at most 45.0%, below Astra's 71.7% ($p = 0.007$). With in-domain training and regime normalization they reached 81.7–86.7% and HGB outperformed Astra ($p = 0.035$);

**Table 4:** Clean-input accuracy (%) of the LLM services (three calls) and of six supervised references. Like-for-like references use the same information as the LLM; the last column gives references that also receive target-domain information (TEP: phase-aware training with sample-40 snapshots; FD002: in-domain training with regime normalization). Ranges are over the six methods. McN: exact McNemar $p$, Astra call 1 versus HGB in the same setting.

| Panel | Sol (calls 1/2/3) | Astra (calls 1/2/3) | Like-for-like | McN | With target info (McN) |
|---|---|---|---|---|---|
| TEP late | 100.0/97.1/100.0 | 100.0/100.0/100.0 | 77.1–100.0 | 1.00 | 78.6–100.0 (1.00) |
| TEP early | 87.1/85.7/88.6 | 88.6/87.1/88.6 | 22.9–54.3 | $1.2 \times 10^{-7}$ | 74.3–88.6 (1.00) |
| FD001 | 71.7/68.3/70.0 | 71.7/73.3/75.0 | 68.3–85.0 | 0.096 | – |
| FD002 | 60.0/26.7/56.7 | 71.7/76.7/78.3 | 33.3–45.0 | 0.007 | 81.7–86.7 (0.035) |

regime normalization alone, estimated from unlabelled FD002 data and applied to the FD001-trained references, gave 71.7–86.7%. The services' advantage therefore came from robustness to conditions absent from the classifier's training data, and it disappeared once the classifiers were given information about those conditions. The services differed sharply in repeatability under operating-condition shift. On FD002, Sol's three calls returned 60.0, 26.7 and 56.7% accuracy with Fleiss' $\kappa = 0.08$; only 10 of 60 engines received the same label in all three calls. Astra's calls were far more consistent ($\kappa = 0.90$; 71.7–78.3%). For Sol, a single FD002 call is thus a poor estimate of what the next call will return.

## 5.2 Descriptor-mapping faults

Table 5 and Figure 3(a) report the mapping contrasts on the 70-run panel; Figure 2 shows every call. The cyclic rotation reduced Sol's accuracy from 99.0% to 40.4% over three calls each ($\Delta = 58.6$ points, 95% CI 37.1–77.1). The effect was large at every level of analysis: 47 of 70 labels changed in the primary call pair (McNemar $p = 1.4 \times 10^{-14}$), the excess disagreement was $E = 0.37$ (95% CI 0.27–0.48), and the call-level randomization test reached its smallest attainable value, $p = 0.10$. Twenty cases were stable differences, correct in all three clean calls and consistently wrong in all three rotated calls. The rotation also made Sol less repeatable: Fleiss' $\kappa$ fell from 0.98 on clean inputs to 0.49.

**Table 5:** Mapping contrasts on 70 source-disjoint TEP runs (three calls per condition). $\Delta$: pooled clean-minus-changed accuracy (percentage points) with 95% crossed bootstrap CI. McN: exact McNemar $p$ for repeat 1, Holm-adjusted over six contrasts. Rand.: two-sided call-level randomization $p$ (20 reassignments; minimum 0.10). $E$: excess disagreement. SD: stable-difference cases. $\kappa$: Fleiss' kappa of the changed condition (clean: Sol 0.98, Astra 1.00).

| Service | Change | Acc. (%) | $\Delta$ [95% CI] | McN $p$ | Rand. $p$ | $E$ | SD | $\kappa$ |
|---|---|---|---|---|---|---|---|---|
| Sol | Cyclic rotation | 40.4 | 58.6 [37.1, 77.1] | $< 10^{-13}$ | 0.10 | 0.371 | 20 | 0.49 |
| Sol | Random 1 | 80.0 | 19.0 [−2.9, 56.7] | 1.00 | 1.00 | −0.003 | 0 | 0.53 |
| Sol | Random 2 | 88.6 | 10.5 [−1.9, 28.6] | 1.00 | 0.70 | 0.002 | 0 | 0.74 |
| Sol | Random 3 | 86.7 | 12.4 [−1.9, 34.3] | $1.5 \times 10^{-7}$ | 0.70 | 0.002 | 0 | 0.69 |
| Sol | Same-unit swap | 90.0 | 9.0 [−1.9, 25.7] | 1.00 | 0.70 | −0.002 | 0 | 0.77 |
| Sol | Intact-pair reorder | 99.0 | 0.0 [−2.9, 2.9] | 1.00 | 1.00 | −0.003 | 0 | 0.98 |
| Astra | All six changes | 100.0 | 0.0 [0.0, 0.0] | 1.00 | 1.00 | 0.000 | 0 | 1.00 |

The random derangements and the same-unit swap behaved differently. Each produced some low-accuracy calls (for example 40.0% in the third call of derangement 1 and 62.9% in the first call of derangement 3), but the other calls of the same input were near-perfect. For derangement 3 the primary-call McNemar test was highly significant (26 changed labels, Holm $p = 1.5 \times 10^{-7}$), yet the pooled interval included zero, the randomization $p$ was 0.70 and no case was a stable difference. A single-call evaluation would have reported this as a strong mapping

effect; the repeated calls show that the damage was confined to individual calls. What the faults did change consistently was repeatability ($\kappa$ 0.53–0.77 against 0.98 for clean inputs). The intact-pair reorder left both accuracy and repeatability unchanged, which confirms that the rotation effect came from the wrong associations rather than from the change in field order.

Astra returned identical, correct labels in all 21 calls on the mapping panel, including every mapping fault. On the preliminary panel it was likewise unaffected by rotation and by descriptor removal. This invariance is useful but should not be read as correct semantic use of the descriptors: a service that ignores the descriptors, relies on value patterns and field positions, or silently repairs a mapping would produce the same output. For the purpose of interface validation, the practical conclusion is that correct labels under a wrong mapping give no evidence that the mapping was checked.

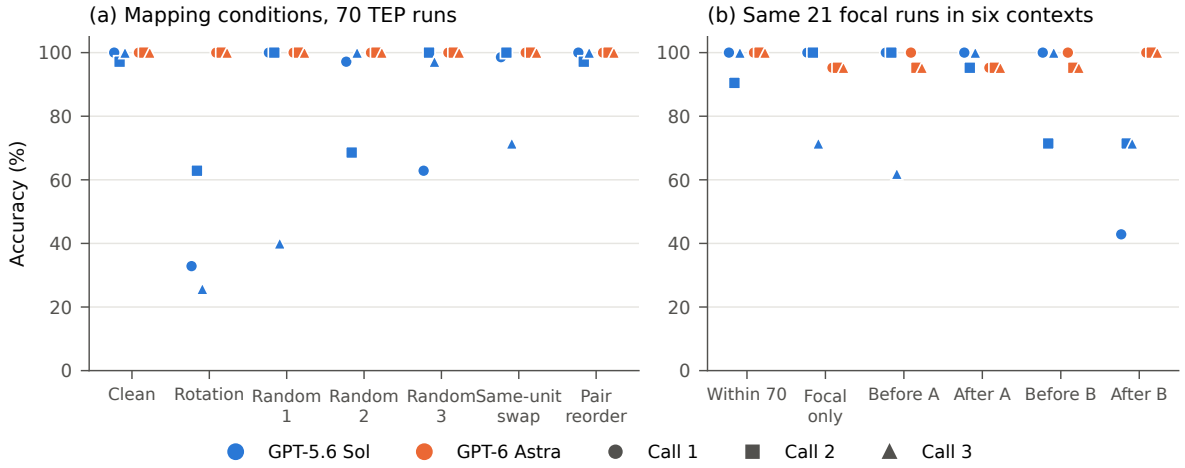



**Figure 2:** All three calls of every mapping condition (a) and every context condition (b) on the source-disjoint TEP panel. Colour identifies the service and marker shape the call. Panel (b) scores only the same 21 focal runs in each request.

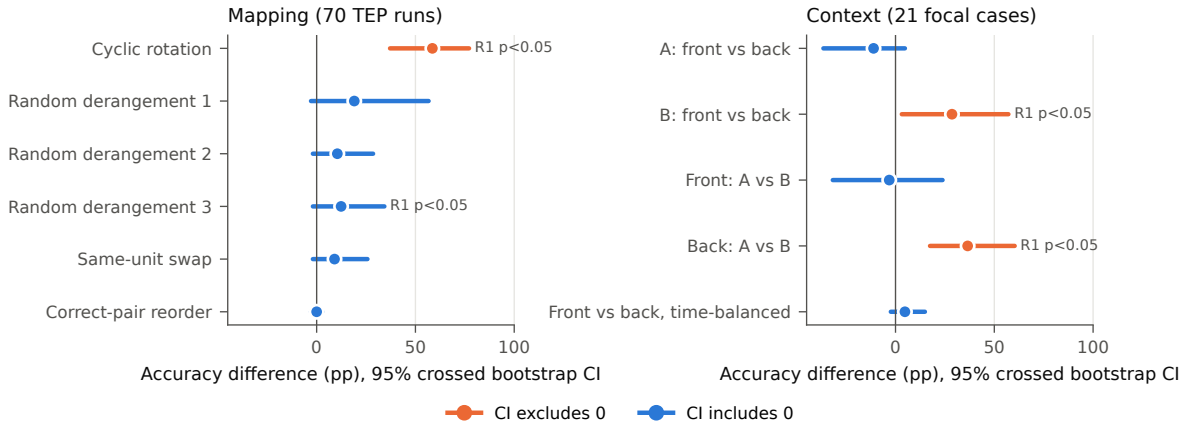



**Figure 3:** Pooled accuracy differences for GPT-5.6 Sol with 95% crossed case-by-call bootstrap intervals. "R1 $p < 0.05$" marks contrasts whose primary-call McNemar test was significant after Holm adjustment. The last row of (b) is the time-balanced ABBA/BAAB replication.

## 5.3 Context, position and call timing

Table 6 reports the four fixed-size context contrasts on the 21 focal runs and the time-balanced replication. Astra's focal labels changed by at most one case in any contrast. For Sol, the

primary calls suggested a strong position effect: the focal runs were all correct when they preceded background B and only 9 of 21 were correct when they followed it (McNemar Holm $p = 0.002$). The same focal records placed after background A were 21/21 correct, giving an equally significant background contrast.

**Table 6:** Context contrasts for Sol on the same 21 focal TEP runs (request size 42). Columns as in Table 5; McN is Holm-adjusted over the four contrasts. The ABBA/BAAB row pools eight calls per order in four blocks (1296 reassignments; minimum two-sided $p = 0.0015$).

| Contrast (left vs right) | Acc. L / R (%) | Δ [95% CI] | McN $p$ | Rand. $p$ | $E$ | SD |
|---|---|---|---|---|---|---|
| A: focal first vs last | 87.3 / 98.4 | −11.1 [−36.5, 4.8] | 1.00 | 1.00 | −0.011 | 0 |
| B: focal first vs last | 90.5 / 61.9 | 28.6 [3.2, 57.1] | 0.002 | 0.30 | 0.063 | 0 |
| Focal first: A vs B | 87.3 / 90.5 | −3.2 [−31.7, 23.8] | 1.00 | 1.00 | −0.063 | 0 |
| Focal last: A vs B | 98.4 / 61.9 | 36.5 [17.5, 60.3] | 0.002 | 0.10 | 0.180 | 0 |
| ABBA/BAAB: first vs last (B) | 98.8 / 94.0 | 4.8 [−2.4, 14.9] | 0.50 | 0.42 | 0.003 | 0 |

The repeated calls do not support a position effect. Sol's two identical "focal after B" calls in repeats 2 and 3 disagreed on 12 of 21 labels while both scored 15/21, so identical-input variation was as large as the primary contrast. The B-position contrast had no stable-difference case, an excess disagreement whose interval included zero, and a randomization $p$ of 0.30. The only contrasts whose bootstrap intervals excluded zero were the two that involved the "focal after B" condition, and all three of its calls were started within the first 12 minutes of the session (Figure 4a). In that early window, Sol's calls with intact mappings averaged 78.2% accuracy (7 calls), against 96.5% for the 11 calls started later. The trend was not significant on its own (Spearman $\rho = 0.28$, $p = 0.26$), but it offers a simpler explanation than a context effect: the low-scoring calls were clustered in time.

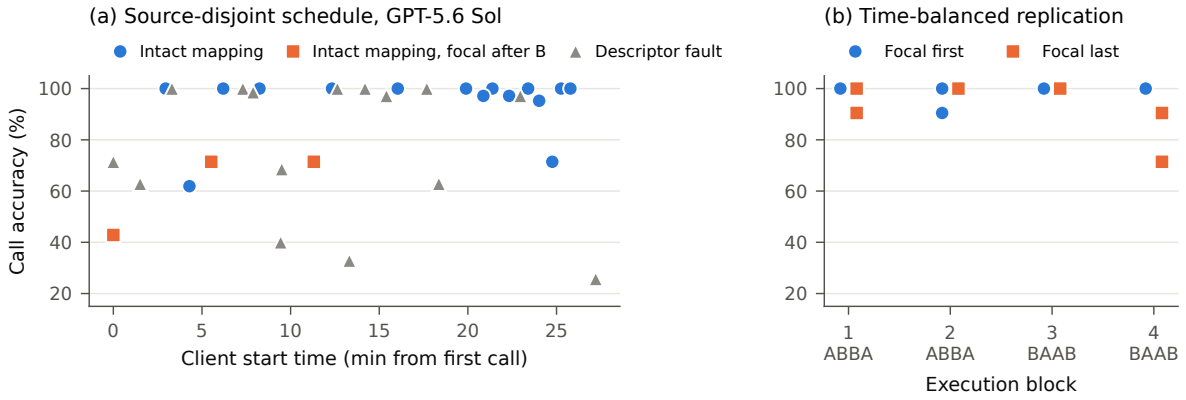



**Figure 4:** (a) Accuracy of every Sol call on the source-disjoint panel against its client start time. Intact-mapping calls (circles, squares) should be near 100% if the input alone determined the output; the three "focal after B" calls (squares) all started early. (b) Focal accuracy in the time-balanced ABBA/BAAB replication; each block contains two calls per order.

The ABBA/BAAB replication tested the B-position contrast directly. Its 16 calls were all valid. Balancing the order within each block removed the systematic difference: focal-first calls averaged 98.8% and focal-last calls 94.0%, with a pooled difference of 4.8 points (95% CI −2.4 to 14.9) and a randomization $p$ of 0.42 over 1,296 reassignments. The largest single discrepancy in the replication, 8 of 21 labels, occurred between the two identical focal-last calls of block 4 (Figure 4b). The earlier 12/21 contrast was therefore not reproduced.

### 5.4 Measured hydraulic test rig

All 63 scheduled calls of the five-block analysis set returned valid responses (1,278 classification slots). Table 7 reports the contrasts and Figure 5 every call. The results differ from TEP in one important respect: on measured data the presentation of the record mattered more than the correctness of the mapping.

With fields in canonical order, Sol classified the 18 focal cycles correctly in 40.0% of cases, and its byte-identical duplicate in 44.4%, close to the 33.3% expected by chance for three balanced classes. These calls were very consistent across blocks, but consistently poor. Reversing the order of the intact descriptor–value pairs, which changes no physical association, raised accuracy to 84.4% ($\Delta = 42.2$ points, 95% CI 17.2–65.6). The call-level randomization test, which only exchanges calls within the same block, gave $p = 0.004$, the smallest value the design can produce. The cyclic descriptor rotation also raised accuracy, to 82.2% ($p = 0.008$). A mapping fault therefore did not reduce accuracy on the rig; both changes to the text layout improved it relative to the canonical order.

**Table 7:** Hydraulic rig: Sol on 18 focal cycles, five execution blocks, one call per condition per block. Where byte-identical duplicates exist they are pooled as the reference. Rand.: two-sided call-level randomization $p$ with exchanges restricted to calls in the same block (243 reassignments when two reference calls are pooled, 32 otherwise). McN: single-call test on block R1, Holm-adjusted over nine contrasts. Negative $\Delta$ means the changed condition was more accurate.

| Contrast (reference vs changed) | Acc. ref / changed (%) | $\Delta$ [95% CI] | McN $p$ | Rand. $p$ |
|---|---|---|---|---|
| Canonical vs intact-pair reversal | 42.2 / 84.4 | −42.2 [−65.6, −17.2] | 0.057 | 0.004 |
| Canonical vs descriptor rotation | 42.2 / 82.2 | −40.0 [−66.7, −15.6] | 1.00 | 0.008 |
| Canonical vs identical duplicate | 40.0 / 44.4 | −4.4 [−23.3, 10.0] | 1.00 | 0.50 |
| Focal+A vs identical duplicate | 58.9 / 62.2 | −3.3 [−31.1, 24.4] | 1.00 | 1.00 |
| A: focal first vs last | 60.6 / 62.2 | −1.7 [−27.8, 22.2] | 0.23 | 1.00 |
| B: focal first vs last | 60.0 / 50.0 | 10.0 [−25.6, 44.4] | 0.094 | 0.88 |
| Focal first: A vs B | 60.6 / 60.0 | 0.6 [−32.2, 32.2] | 0.094 | 1.00 |
| Focal last: A vs B | 62.2 / 50.0 | 12.2 [−17.8, 42.2] | 0.23 | 0.50 |
| Focal only vs focal+A | 42.2 / 60.6 | −18.3 [−37.2, −3.9] | 1.00 | 0.028 |

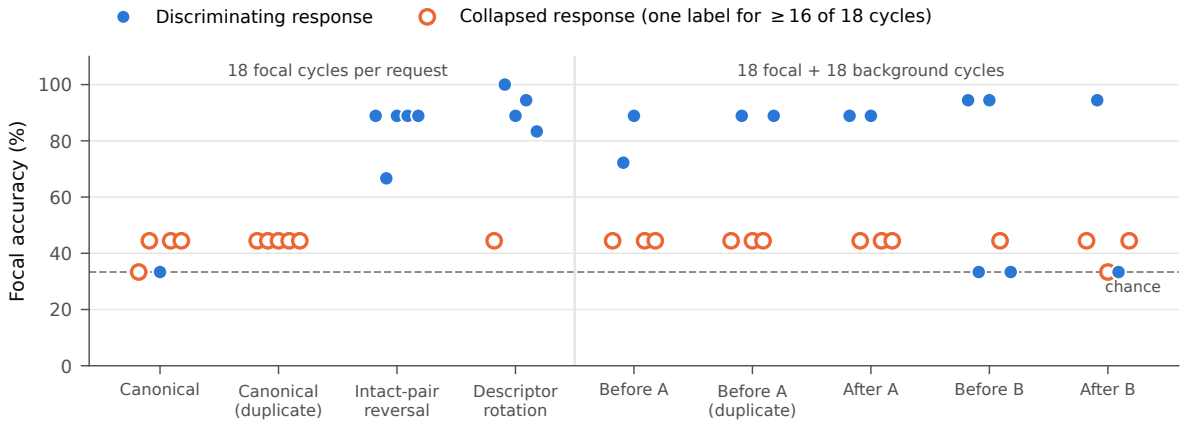



**Figure 5:** Every Sol call on the measured hydraulic rig; the five points per condition are execution blocks R1–R5 from left to right. Open markers are collapsed responses that assign one label to at least 16 of the 18 focal cycles. Dashed line: chance level for three balanced classes.

The per-call outputs explain the pattern. Sol's responses fell into two modes (Figure 5). In a *collapsed* response it assigned one label, almost always "no leakage", to at least 16 of the 18 focal cycles; such calls averaged 43.5% accuracy (23 calls). Otherwise the labels were spread

over the three classes, and 18 of these 22 calls scored 12–18 correct. Collapse occurred in 9 of 10 canonical calls, in none of the 5 reversed calls (Fisher exact $p = 0.002$) and in 1 of 5 rotated calls ($p = 0.017$). It is a whole-request failure, and its frequency depended on the text layout rather than on the physical content.

Context changes on the rig behaved as they had on TEP once calls were pooled. In block R1 all four fixed-size context contrasts had single-call McNemar $p < 0.05$ before adjustment, yet none survived Holm adjustment and none had a randomization $p$ below 0.5. With 36 records per request, byte-identical calls alternated between about 8 and 16 correct focal cycles depending on whether the response collapsed, which produced the apparent single-call effects. Adding background A to the focal records raised accuracy from 42.2% to 60.6% ($p = 0.028$), mostly by lowering the collapse rate (9/10 against 13/25; $p = 0.055$), but this comparison changes request size and composition together. The 18 single-record calls returned 7 correct labels (38.9%).

The analysed hydraulic set used 1.24 million input-plus-output tokens. Median latency was 49 s per call (range 16–263 s), and even a single-record request consumed about 11,000 tokens because of the fixed instruction overhead of each fresh CLI session.

### 5.5 Countermeasures: gateway, fixed panels and voting

On all 54 measured hydraulic records, the trusted-channel gateway accepted every clean input and every intact-pair reversal and wrote them in identical canonical form. It rejected all 54 cyclic rotations and all 54 same-unit descriptor swaps with no false rejection of clean inputs. It accepted all 54 hidden value swaps, because a value moved together with a consistent descriptor is indistinguishable from a real measurement at this layer. Rejected records receive no automated diagnosis, so the gateway converts wrong answers into withheld answers; its benefit depends on how rejected requests are handled downstream.

Canonical ordering makes an intact-pair reorder byte-identical to the clean request, so the reorder can no longer change the output. This removes one source of variation but does not make the output correct: on the hydraulic rig, the canonical order was the layout that most often produced collapsed responses. A fixed panel also does nothing about call-to-call variation, which persisted for byte-identical requests in every study.

Figure 6 shows three-call majority voting on the TEP panels. Voting restored Sol's accuracy under the random derangements and the same-unit swap from single-call means of 80.0–90.0% to 98.6–100%, and under four of the five context conditions to 100%. It did not help where the errors were systematic: 44.3% under cyclic rotation and 42.9% for the early "focal after B" calls. On FD002 voting raised Sol from a single-call mean of 47.8% to 60.0%. The cost is three requests per decision. Voting also yields a free diagnostic: Sol's three calls disagreed on 2.9% of cases with clean inputs and on 29–60% of cases under every mapping fault, so disagreement between repeated calls flags a corrupted request for this service. The signal is absent for Astra, which was fully consistent under all mapping faults.

## 6 Discussion

### 6.1 Implications for LLM diagnostic interfaces

The results separate three properties that a single accuracy figure merges. *Channel integrity* concerns whether each value carries the right description. Sol responded strongly to one mapping fault, Astra to none, and both behaviours are compatible with high clean accuracy. The mapping therefore has to be verified outside the model, against a registry of trusted acquisition IDs, and the verification has to be tested with injected faults, including faults that keep units consistent. Our gateway detected every visible descriptor fault but none of the hidden value swaps, which marks the boundary of what a metadata check can do.

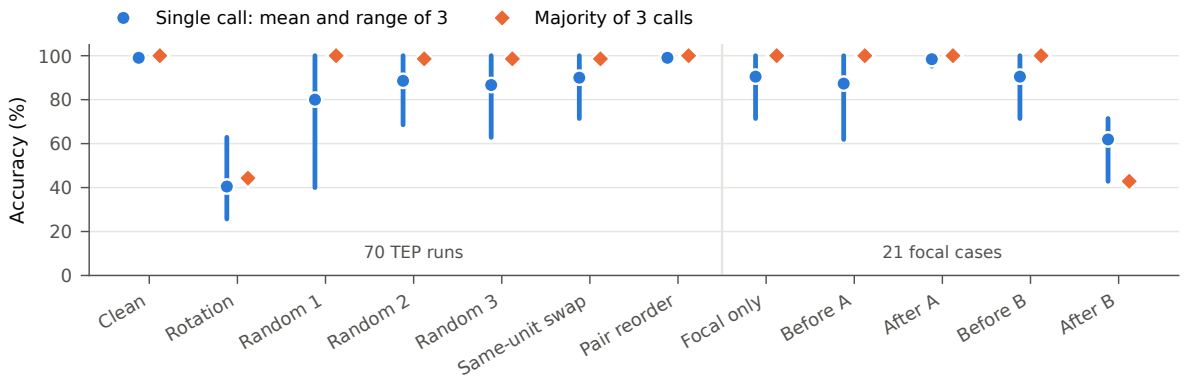



**Figure 6:** Single-call accuracy (mean and range of three calls) and three-call majority-vote accuracy for GPT-5.6 Sol on the TEP mapping panel (left) and the 21 focal cases in five contexts (right).

*Decision reproducibility* concerns whether the same record receives the same label. Here the dominant factor in our data was not the input change but the call: identical requests differed by up to 12 of 21 labels, one service's FD002 labels were almost unrepeatable, and the low-accuracy calls clustered in time. Context and position effects reported from single calls should be treated with caution. In our study, time-blocked schedules with repeated controls were necessary to distinguish an input effect from a service-period effect, and the randomization test gives a valid $p$-value only when enough calls are available; three unblocked calls per condition cannot produce $p < 0.10$.

The hydraulic study adds a caution about *presentation*. On TEP the intact-pair reorder was harmless, but on the measured rig the canonical order produced collapsed, near-chance responses and the reversed order did not. Field order is therefore a design parameter to be validated per asset and service, not a neutral formatting choice, and canonicalization should fix an order that has been shown to work. The collapse mode is reminiscent of the majority-label and recency biases reported for in-context learning [35]; a simple guard is to flag any response whose label distribution is far more concentrated than the expected class prevalence.

*Output availability* concerns whether the interface returns a usable decision at all. Whole-request validation made failures explicit: time-outs, incomplete arrays and output-budget exhaustion removed decisions that a lenient parser might have recovered or silently truncated. Conditional accuracy and availability should be reported together.

### 6.2 Recommended evaluation protocol

From these observations we suggest a minimal protocol for evaluating an LLM diagnostic interface before deployment: (1) include at least one cyclic mapping fault, one same-unit swap and one intact-pair reorder alongside clean inputs; (2) submit every condition at least five times in time blocks that contain all conditions; (3) report single-call, pooled and call-level results together, with Fleiss' $\kappa$ of identical calls; (4) validate channel mappings outside the model and test the validator with injected faults; and (5) report availability, token usage and latency with accuracy. The protocol is inexpensive relative to a deployment decision: the complete five-block hydraulic study used about 1.2 million tokens and about 33 minutes of wall-clock time with two concurrent calls.

### 6.3 Limitations

The study has several limitations. Only two services were evaluated in full, both through one command-line interface without temperature control, and the requested aliases do not identify fixed model weights; other services and settings may behave differently. The simulated panels

are balanced and use late or fixed-time snapshots, and the retrospective RUL classes are not maintenance thresholds. The measured data come from a single laboratory rig, so the hydraulic results do not establish cross-device or field performance. Public exposure of TEP and C-MAPSS during pre-training is unknown. The ABBA/BAAB replication and the hydraulic study were designed after the source-disjoint results and are exploratory; voting and gateway assessments were post hoc. Finally, the call-level tests are exact but coarse with three calls per condition, and the crossed bootstrap is approximate with so few calls. Larger, preregistered designs with more services and more time blocks would allow effect estimates across service periods.

# 7 Conclusion

We evaluated LLM-based industrial fault diagnosis with repeated calls, crossing descriptor-mapping faults, meaning-preserving reorderings and context changes with identical-input repeats on simulated and measured data. A cyclic descriptor rotation caused a large and repeatable accuracy loss for one service, while another service gave correct labels under every mapping fault; high accuracy therefore does not show that sensor associations are used or checked. A position effect that was highly significant in a single-call test was not reproduced under a time-balanced schedule, and a mapping effect with a single-call $p < 10^{-6}$ fell within call-to-call variation. On a measured hydraulic rig, the order of correctly mapped fields decided whether the service collapsed to a single label, doubling accuracy between two meaning-equivalent layouts. For deployment, channel mappings should be verified by an independent gateway, evaluations should use repeated, time-blocked calls and report repeatability and availability alongside accuracy, and voting should be reserved for transient rather than systematic errors. Future work should extend the protocol to more services, open-weight models with controllable decoding, and multiple physical assets.

## CRediT authorship contribution statement

**Song Ouyang:** Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Visualization, Writing – original draft, Writing – review & editing.

## Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## Data availability

All data used are publicly available: the TEP simulation data [57], the NASA C-MAPSS data [61] and the UCI hydraulic condition-monitoring data [62]. Fixed model inputs, prompts, frozen model outputs, execution logs, per-case predictions, baseline code and the statistical analysis scripts are available at `https://github.com/song-ouyang/sci`.

## Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used OpenAI Codex and Anthropic Claude to assist with code maintenance, consistency checks and language editing. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication. The LLM services evaluated in this study are the object of the research and are described in Section 4.

## References


[1] Yaguo Lei, Bin Yang, Xinwei Jiang, Feng Jia, Naipeng Li, and Asoke K Nandi. Applications of machine learning to machine fault diagnosis: A review and roadmap. *Mechanical Systems and Signal Processing*, 138:106587, 2020. doi: 10.1016/j.ymssp.2019.106587. URL https://doi.org/10.1016/j.ymssp.2019.106587.

[2] Rui Zhao, Ruqiang Yan, Zhenghua Chen, Kezhi Mao, Peng Wang, and Robert X Gao. Deep learning and its applications to machine health monitoring. *Mechanical Systems and Signal Processing*, 115: 213–237, 2019. doi: 10.1016/j.ymssp.2018.05.050. URL https://doi.org/10.1016/j.ymssp.2018.05.050.

[3] Hamzah A. A. M. Qaid, Bo Zhang, Shuai Su, Dan Li, See-Kiong Ng, and Wei Li. Large language models for explainable fault diagnosis of machines. *Engineering Applications of Artificial Intelligence*, 163:113131, 2026. doi: 10.1016/j.engappai.2025.113131. URL https://www.sciencedirect.com/science/article/pii/S0952197625031628.

[4] Shuwen Zheng, Kai Pan, Jie Liu, and Yunxia Chen. Empirical study on fine-tuning pre-trained large language models for fault diagnosis of complex systems. *Reliability Engineering & System Safety*, 252: 110382, 2024. doi: 10.1016/j.ress.2024.110382. URL https://www.sciencedirect.com/science/article/abs/pii/S095183202400454X.

[5] Yan-Fu Li, Huan Wang, and Muxia Sun. ChatGPT-like large-scale foundation models for prognostics and health management: A survey and roadmaps. *Reliability Engineering & System Safety*, 243: 109850, 2024. doi: 10.1016/j.ress.2023.109850. URL https://doi.org/10.1016/j.ress.2023.109850.

[6] Zhoujun Cheng, Jungo Kasai, and Tao Yu. Batch prompting: Efficient inference with large language model APIs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 792–810. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-industry.74. URL https://aclanthology.org/2023.emnlp-industry.74/.

[7] Jianzhe Lin, Maurice Diesendruck, Liang Du, and Robin Abraham. BatchPrompt: Accomplish more with less. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=Agyicd577r.

[8] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):1–28, 2025. doi: 10.1145/3697010. URL https://doi.org/10.1145/3697010.

[9] Berk Atıl, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. Non-determinism of "deterministic" LLM system settings in hosted environments. In *Proceedings of the 5th Workshop on Evaluation and Comparison of NLP Systems (Eval4NLP)*, pages 135–148, Mumbai, India, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.eval4nlp-1.12. URL https://aclanthology.org/2025.eval4nlp-1.12/.

[10] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. Tabllm: Few-shot classification of tabular data with large language models. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 5549–5581. PMLR, 2023. URL https://proceedings.mlr.press/v206/hegselmann23a.html.

[11] Hejia Liu, Mochen Yang, and Gediminas Adomavicius. Robustness is important: Limitations of LLMs for predictions on tabular data. *PNAS Nexus*, 5(6):pgag197, 2026. doi: 10.1093/pnasnexus/pgag197.

[12] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *The Twelfth International Conference on Learning Representations (ICLR)*, pages 25055–25083, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/6c0e99d736da621403018ca7b32b1a4d-Abstract-Conference.html.

[13] Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. State of what art? A call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 12:933–949, 2024. doi: 10.1162/tacl_a_00681. URL https://aclanthology.org/2024.tacl-1.52/.

[14] Laifa Tao, Shangyu Li, Haifei Liu, Qixuan Huang, Liang Ma, Guoao Ning, Yiling Chen, Yunlong Wu, Bin Li, Weiwei Zhang, et al. An outline of prognostics and health management large model: Concepts, paradigms, and challenges. *Mechanical Systems and Signal Processing*, 232:112683, 2025. doi: 10.1016/j.ymssp.2025.112683. URL https://doi.org/10.1016/j.ymssp.2025.112683.

[15] Xuefeng Chen, Yaguo Lei, Yanfu Li, Simon Parkinson, Xiang Li, Jinxin Liu, Fan Lu, Huan Wang, Zisheng Wang, Bin Yang, Shilong Ye, and Zhibin Zhao. Large models for machine monitoring and fault diagnostics: Opportunities, challenges, and future direction. *Journal of Dynamics, Monitoring and Diagnostics*, 4(2):76–90, 2025. doi: 10.37965/jdmd.2025.832. URL https://doi.org/10.37965/jdmd.2025.832.

[16] Laifa Tao, Haifei Liu, Guoao Ning, Wenyan Cao, Bohao Huang, and Chen Lu. LLM-based framework for bearing fault diagnosis. *Mechanical Systems and Signal Processing*, 224:112127, 2025. doi: 10.1016/j.ymssp.2024.112127. URL https://doi.org/10.1016/j.ymssp.2024.112127.

[17] Hamzah A. A. M. Qaid, Bo Zhang, Dan Li, See-Kiong Ng, and Wei Li. FD-LLM: Large language model for fault diagnosis of machines. arXiv preprint arXiv:2412.01218, 2024. URL https://arxiv.org/abs/2412.01218.

[18] Shuxin Lin, Dhaval C Patel, and Christodoulos Constantinides. Fine-tuned thoughts: Leveraging chain-of-thought reasoning for industrial asset health monitoring. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 20687–20700, Suzhou, China, November 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-emnlp.1126. URL https://aclanthology.org/2025.findings-emnlp.1126/.

[19] Jiaxin Ren, Xue Liu, Tianlei Wang, Zhibin Zhao, Xuefeng Chen, Weihua Li, and Ruqiang Yan. PHM–GPT: A large language model for prognostics and health management. *Engineering*, 63:156–172, 2026. doi: 10.1016/j.eng.2025.11.001. URL https://doi.org/10.1016/j.eng.2025.11.001.

[20] Abdullah Khan, Rahul Nahar, Hao Chen, Gonzalo E. Constante Flores, and Can Li. FaultExplainer: Leveraging large language models for interpretable fault detection and diagnosis. *Computers & Chemical Engineering*, 199:109152, 2025. doi: 10.1016/j.compchemeng.2025.109152. URL https://doi.org/10.1016/j.compchemeng.2025.109152.

[21] Kahiomba Sonia Kiangala and Zenghui Wang. Explainable LLM-assisted fault diagnosis for smart manufacturing using multi-trace diagnostic rationales and verification. *The International Journal of Advanced Manufacturing Technology*, 2026. doi: 10.1007/s00170-026-19043-z. Published online 17 September 2026.

[22] Christodoulos Constantinides, Dhaval Patel, Shuxin Lin, Claudio Guerrero, Sunil Patil, and Jayant Kalagnanam. FailureSensorIQ: A multi-choice QA dataset for understanding sensor relationships and failure modes. In *Advances in Neural Information Processing Systems*, volume 38, pages 140021–140060. Curran Associates, Inc., 2025. doi: 10.52202/085713-4212. URL https://proceedings.nips.cc/paper_files/paper/2025/hash/b7ba530076dee3b4f19847d8dce983da-Abstract-Datasets_and_Benchmarks_Track.html. Datasets and Benchmarks Track.

[23] Pengrui Quan, Xiaomin Ouyang, Jeya Vikranth Jeyakumar, Ziqi Wang, Yang Xing, and Mani Srivastava. SensorBench: Benchmarking LLMs in coding-based sensor processing. In *Proceedings of the 26th International Workshop on Mobile Computing Systems and Applications*, HotMobile '25, pages 25–30. Association for Computing Machinery, 2025. doi: 10.1145/3708468.3711882. URL https://doi.org/10.1145/3708468.3711882.

[24] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. Time-LLM: Time series forecasting by reprogramming large language models. In *The Twelfth International Conference on Learning Representations*, pages 23857–23880, 2024.

[25] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[26] Xiyuan Zhang, Ranak Roy Chowdhury, Rajesh K. Gupta, and Jingbo Shang. Large language models for time series: A survey. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 8335–8343, 2024. doi: 10.24963/ijcai.2024/921.

[27] Mingtian Tan, Mike A. Merrill, Vinayak Gupta, Tim Althoff, and Thomas Hartvigsen. Are language models actually useful for time series forecasting? In *Advances in Neural Information Processing Systems*, volume 37, pages 60162–60191, 2024. doi: 10.52202/079017-1922. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/6ed5bf446f59e2c6646d23058c86424b-Abstract-Conference.html.

[28] Mike A. Merrill, Mingtian Tan, Vinayak Gupta, Thomas Hartvigsen, and Tim Althoff. Language models still struggle to zero-shot reason about time series. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3512–3533, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.201. URL https://aclanthology.org/2024.findings-emnlp.201/.

[29] Ming Jin, Yifan Zhang, Wei Chen, Kexin Zhang, Yuxuan Liang, Bin Yang, Jindong Wang, Shirui Pan, and Qingsong Wen. Position: What can large language models tell us about time series analysis. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 22260–22276. PMLR, 2024. URL https://proceedings.mlr.press/v235/jin24i.html.

[30] Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy-yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. LIFT: Language-interfaced fine-tuning for non-language machine learning tasks. In *Advances in Neural Information Processing Systems*, volume 35, pages 11763–11784, 2022. doi: 10.52202/068431-0855. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/4ce7fe1d2730f53cb3857032952cd1b8-Abstract-Conference.html.

[31] Vadim Borisov, Kathrin Seßler, Tobias Leemann, Martin Pawelczyk, and Gjergji Kasneci. Language models are realistic tabular data generators. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=cEygmQNOeI.

[32] Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. Table meets LLM: Can large language models understand structured table data? A benchmark and empirical study. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM '24)*, pages 645–654. Association for Computing Machinery, 2024. doi: 10.1145/3616855.3635752. URL https://doi.org/10.1145/3616855.3635752.

[33] Xi Fang, Weijie Xu, Fiona Anting Tan, Ziqing Hu, Jiani Zhang, Yanjun Qi, Srinivasan H. Sengamedu, and Christos Faloutsos. Large language models (LLMs) on tabular data: Prediction, generation, and understanding – A survey. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=IZnrCGF9WI.

[34] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8086–8098, Dublin, Ireland, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.556. URL https://aclanthology.org/2022.acl-long.556/.

[35] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 12697–12706. PMLR, 2021. URL https://proceedings.mlr.press/v139/zhao21c.html.

[36] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *The Twelfth International Conference on Learning Representations (ICLR)*, pages 19426–19454, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/54dd9e0cff6d9214e20d97eb2a3bae49-Abstract-Conference.html.

[37] Pouya Pezeshkpour and Estevam Hruschka. Large language models sensitivity to the order of options in multiple-choice questions. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2006–2017, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-naacl.130. URL https://aclanthology.org/2024.

findings-naacl.130/.

[38] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9440–9450, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.511. URL https://aclanthology.org/2024.acl-long.511/.

[39] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://aclanthology.org/2024.tacl-1.9/.

[40] Cheng-Yu Hsieh, Yung-Sung Chuang, Chun-Liang Li, Zifeng Wang, Long Le, Abhishek Kumar, James Glass, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. Found in the middle: Calibrating positional attention bias improves long context utilization. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14982–14995, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.890. URL https://aclanthology.org/2024.findings-acl.890/.

[41] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227. PMLR, 2023. URL https://proceedings.mlr.press/v202/shi23a.html.

[42] Meihao Fan, Xiaoyue Han, Ju Fan, Chengliang Chai, Nan Tang, Guoliang Li, and Xiaoyong Du. Cost-effective in-context learning for entity resolution: A design space exploration. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 3696–3709. IEEE, 2024. doi: 10.1109/ICDE60146.2024.00284. URL https://dbgroup.cs.tsinghua.edu.cn/ligl/papers/ICDE24-ER.pdf.

[43] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442. URL https://aclanthology.org/2020.acl-main.442/.

[44] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=1PL1NIMMrw.

[45] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=VD-AYtP0dve.

[46] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *The Twelfth International Conference on Learning Representations (ICLR)*, pages 23650–23678, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/6733cf15e10e2cd1d59af033c3bb8507-Abstract-Conference.html.

[47] Potsawee Manakul, Adian Liusie, and Mark Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9004–9017, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.557. URL https://aclanthology.org/2023.emnlp-main.557/.

[48] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998. doi: 10.1162/089976698300017197. URL https://doi.org/10.1162/089976698300017197.

[49] R. H. Baayen, D. J. Davidson, and D. M. Bates. Mixed-effects modeling with crossed random effects for subjects and items. *Journal of Memory and Language*, 59(4):390–412, 2008. doi: 10.1016/j.jml.2007.12.005. URL https://doi.org/10.1016/j.jml.2007.12.005.

[50] Douglas Bates, Martin Mächler, Ben Bolker, and Steve Walker. Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, 67(1):1–48, 2015. doi: 10.18637/jss.v067.i01. URL

https://doi.org/10.18637/jss.v067.i01.

[51] Art B. Owen. The pigeonhole bootstrap. *The Annals of Applied Statistics*, 1(2):386–411, 2007. doi: 10.1214/07-AOAS122.

[52] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

[53] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[54] Eugene S. Edgington and Patrick Onghena. *Randomization Tests*. Chapman & Hall/CRC, Boca Raton, FL, 4th edition, 2007. doi: 10.1201/9781420011814.

[55] Joseph L. Fleiss. Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5):378–382, 1971. doi: 10.1037/h0031619.

[56] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[57] Cory A. Rieth, Ben D. Amsel, Randy Tran, and Maia B. Cook. Additional Tennessee Eastman Process simulation data for anomaly detection evaluation. Harvard Dataverse, 2017. URL https://doi.org/10.7910/DVN/6C3JR1. Dataset, version 1.0.

[58] James J Downs and Ernest F Vogel. A plant-wide industrial process control problem. *Computers & Chemical Engineering*, 17(3):245–255, 1993. doi: 10.1016/0098-1354(93)80018-I. URL https://doi.org/10.1016/0098-1354(93)80018-I.

[59] Afrânio Melo, Maurício M. Câmara, Nayher Clavijo, and José Carlos Pinto. Open benchmarks for assessment of process monitoring and fault diagnosis techniques: A review and critical analysis. *Computers & Chemical Engineering*, 165:107964, 2022. doi: 10.1016/j.compchemeng.2022.107964. URL https://doi.org/10.1016/j.compchemeng.2022.107964.

[60] Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management*, pages 1–9. IEEE, 2008. doi: 10.1109/PHM.2008.4711414. URL https://doi.org/10.1109/PHM.2008.4711414.

[61] Abhinav Saxena and Kai Goebel. Turbofan engine degradation simulation data set. NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, 2008. URL https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/. Data set.

[62] Nikolai Helwig, Eliseo Pignanelli, and Andreas Schütze. Condition monitoring of hydraulic systems. UCI Machine Learning Repository, 2015. URL https://doi.org/10.24432/C5CW21. Dataset; accessed 28 September 2026; CC BY 4.0.

[63] Nikolai Helwig, Eliseo Pignanelli, and Andreas Schütze. Condition monitoring of a complex hydraulic system using multivariate statistics. In *2015 IEEE International Instrumentation and Measurement Technology Conference (I2MTC) Proceedings*, pages 210–215. IEEE, 2015. doi: 10.1109/I2MTC.2015.7151267. URL https://doi.org/10.1109/I2MTC.2015.7151267.

[64] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[65] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[66] Emmanuel Ramasso and Abhinav Saxena. Performance benchmarking and analysis of prognostic methods for CMAPSS datasets. *International Journal of Prognostics and Health Management*, 5(2), 2014. doi: 10.36001/ijphm.2014.v5i2.2236. URL https://papers.phmsociety.org/index.php/ijphm/article/view/2236.